EXAMEN FINAL · DESPLIEGUE DEL PROYECTO

VEKTORA

Cómo cumplimos cada punto de la propuesta.

Nicolás Bentancour · Agustín Perdomo · Matías Piñeyro · Thiago Rodríguez

Programación Avanzada · UTU · Docente: Matias Alfaro

LA IDEA

Ninguna persona decide nada en el ciclo.

Dividir, asignar y aceptar lo hacen agentes de IA. Cada decisión queda guardada con su explicación.

CÓMO FUNCIONA

Seis pasos, una sola persona al principio

1 · Persona

Escribe qué quiere

Una frase y un presupuesto.

2 · IA planificadora

Lo parte en tareas

Con requisitos, orden y presupuesto.

3 · Sistema

Arma el grafo

Qué va antes de qué.

4 · Matching

Elige quién la hace

Por habilidades y reputación.

5 · Proveedor

Entrega con evidencia

Una prueba por requisito.

6 · IA jueza

Revisa y desbloquea

Si aprueba, siguen las próximas.

LO QUE PEDÍA LA PROPUESTA

Los seis criterios, uno por uno

Criterio	Pts	Cómo lo cumplimos	Estado
Requisitos R1–R12	15	Los 12, con su archivo en el README	Hecho
Nivel 1: Docker	20	Dockerfile, compose, .env e imagen en ghcr.io	Hecho
Nivel 2: nube	15	Base en Aiven con SSL y app en Render con HTTPS	Hecho
Nivel 3: Kubernetes	15	minikube con Secret, ConfigMap y probes	Hecho
Seguridad	15	Los 4 análisis, corregidos y explicados	Hecho
Documentación	20	README, guía, diagramas e informe	Faltan 4 capturas

ARQUITECTURA

La misma que pedía la propuesta



GitHub Actions
pruebas y análisis en cada cambio, y el reloj que despierta a los agentes

REQUISITOS · 1 DE 3

Datos, CRUD, usuarios y capas

Id	Pide	Cómo lo cumplimos	Dónde
R1	Modelo de datos	14 tablas. Proyecto → tareas es 1:N; proveedor ↔ habilidad es N:M	db/schema.sql
R2	CRUD	Alta, baja, cambio y consulta de proyectos, cuentas y habilidades	app/acciones/proyectos.ts, lib/admin
R3	Usuarios y roles	Contraseñas con BCrypt y 4 roles: cliente, proveedor, ambos y admin	lib/auth/password.ts, lib/auth/permisos.ts
R4	Capas	Pantallas → servicio → DAO (interfaz + PostgreSQL) → base	app/, lib/*/repository.ts

REQUISITOS · 2 DE 3

Objetos, consultas y errores

Id	Pide	Cómo lo cumplimos	Dónde
R5	Herencia e interfaz	Errores que heredan de VektoraError; la interfaz de IA tiene dos versiones: Groq y Gemini	lib/errores, lib/ai/providers
R6	Consultas	Búsqueda con filtros y JOIN; reporte con COUNT, SUM, AVG y GROUP BY	lib/consultas/busqueda.ts, reporte.ts
R7	Transacción	Entregar toca 3 tablas: todo se guarda junto o nada (commit o rollback)	lib/entregas, lib/db/client.ts
R8	Validaciones	Zod revisa cada entrada; los errores internos nunca llegan a la pantalla	lib/validacion, lib/errores

REQUISITOS · 3 DE 3

Seguridad, configuración y estado

Id	Pide	Cómo lo cumplimos	Dónde
R9	Consultas seguras	Siempre $1, $2… con los datos aparte: el equivalente a PreparedStatement	lib/db/client.ts
R10	Config. externa	DB_HOST, DB_PORT, DB_NAME, DB_USER y DB_PASSWORD salen del entorno	lib/db/config.ts, .env.example
R11	Datos de prueba	Script de tablas y script de datos: 30 habilidades, 6 cuentas, 8 tareas…	sql/01-crear-tablas.sql, sql/02-datos-prueba.sql
R12	/health	200 si la app y la base responden en menos de 3 s; si no, 503	app/health/route.ts

NIVEL 1 · OBLIGATORIO · 20 PUNTOS

Todo en contenedores Docker

1 **Dockerfile** en 3 etapas: la imagen final no lleva el código fuente y corre sin root

2 **docker-compose.yml**: base PostgreSQL y app; la base se carga sola con sql/

3 **Credenciales en .env**, nunca en el código

4 **.env en .gitignore**; .env.example solo con los nombres

5 **Imagen en ghcr.io**: se prueba con compose y Trivy antes de publicarse

Evidencia

docker compose up levanta todo con un solo comando.

GitHub lo prueba desde cero en cada cambio.

Captura 4 (ghcr.io): lista. Captura 1: falta sacarla en una PC.

NIVEL 2 · LA NUBE · 15 PUNTOS

Base en Aiven, app en Render

1 **Base PostgreSQL 16 en Aiven**, plan gratis

2 **Scripts de sql/ ejecutados contra Aiven** con npm run sql:aplicar

3 **Conexión por variables** (host, puerto, base, usuario, contraseña) y **SSL** que verifica el certificado de Aiven

4 **Render construye desde el Dockerfile**; las variables van en su panel

Evidencia

vektora.onrender.com responde por HTTPS.

Se crea una cuenta en la app y se ve con un SELECT en Aiven.

Captura 6 (Render): lista. Captura 5 (Aiven): falta.

NIVEL 3 · KUBERNETES · 15 PUNTOS

App y base orquestadas en minikube

1 **Deployments** de app y base, con imagen, variables, requests y limits

2 **Services**: la base solo por dentro; la app por el puerto 30080

3 **Secret** con las contraseñas y **ConfigMap** con el resto

4 **Probes** sobre /health; la base, con pg_isready

\+ Un volumen para que los datos sobrevivan a un reinicio

Evidencia

kubectl get pods,svc con todo en Running.

GitHub lo repite en un clúster kind en cada cambio.

Captura 7: lista.

SEGURIDAD (SSDLC) · 15 PUNTOS

Pedían 3 análisis: hicimos los 4, y uno más

Análisis	Herramienta	Antes	Después
SCA · librerías	npm audit y Dependabot	6	0
SAST · código	CodeQL	3 altas	0
Imagen	Trivy	31	0
DAST · web	OWASP ZAP, solo a nuestra URL	10	3 explicadas
Extra · secretos	gitleaks, todo el historial	3 falsas	0

Cada hallazgo, su corrección o su justificación está en docs/informe-seguridad.md.

HARDENING MÍNIMO

Los cuatro puntos, cumplidos

Sin contraseñas en el repositorio

Ni en el código, ni en el Dockerfile, ni en el historial: gitleaks lo revisa en cada cambio.

Contenedor sin root

La app corre como el usuario vektora (uid 1001), en Docker y en Kubernetes.

Aiven solo por SSL

DB_SSL=require y el certificado de Aiven verificado con DB_SSL_CA.

R3 y R9

BCrypt de coste 12 para las contraseñas y consultas siempre parametrizadas.

ENTREGABLES · DOCUMENTACIÓN 20 PUNTOS

El repositorio, como lo pedía la propuesta

```
src/ → app/ + lib/ + components/
sql/01-crear-tablas.sql
sql/02-datos-prueba.sql
Dockerfile · docker-compose.yml
.env.example
k8s/ app, base, secret, configmap
docs/guia-despliegue.md
docs/informe-seguridad.md
docs/diagramas/
README.md (tabla R1–R12)
```

¿Y src/?

Next.js exige carpetas con nombre fijo. Lo que sería src/ son app/, lib/ y components/.

De más

Guía para la defensa, guía en palabras simples y los documentos teóricos 01 a 07.

Reproducible

La guía de despliegue dice qué hacer, qué se tiene que ver y qué hacer si no se ve.

ANEXO · LISTA DE CONTROL

Dónde estamos, punto por punto

Hecho · R1–R10 y tabla en el README

Hecho · compose desde cero (en GitHub)

Hecho · sin contraseñas en el historial

Hecho · .env.example y .env ignorado

Hecho · imagen en ghcr.io

Hecho · Aiven por SSL

Hecho · Render por HTTPS

Hecho · k8s/ y pods en Running

Hecho · análisis de seguridad

Hecho · README, guía, diagramas, informe

Falta · guía probada por otro grupo

Falta · ensayar: todos explican todo

Capturas que faltan en la guía: 1, 2 y 3 (Docker en una PC), 5 (Aiven) y la de SonarLint.

DEFENSA ORAL · PRUEBA EN VIVO

Lo que vamos a mostrar

1. **Inicio de sesión** en vektora.onrender.com, con el candado de HTTPS
2. **Alta de un dato**: crear una cuenta nueva desde la app
3. **Consulta en la base**: esa cuenta, con un SELECT en Aiven
4. **Arquitectura y R1–R12**: el diagrama y cada requisito en el código
5. **Preguntas**: cada uno puede explicar cualquier archivo

PLANES GRATIS

Sus límites, y cómo los manejamos

Límite	Qué hacemos
Render se duerme a los 15 min	Abrir /health antes de la defensa
Render no tiene tareas programadas	GitHub Actions hace de reloj de los agentes
Aiven apaga la base sin uso	Encenderla desde la consola antes
Kubernetes en la nube cuesta	minikube en la PC y kind en GitHub
La IA gratis tiene cortes y cuotas	Groq y, si falla, Gemini; cuotas por usuario

Costo total del proyecto: **0 USD**, sin tarjeta de crédito.

VEKTORA

¿Preguntas?

Gracias por escucharnos.

vektora.onrender.com

github.com/Nicrolax/_JBDPA_Grupo_1